Temporal Label Models for Estimating Progress in LLM Agent Trajectories

Anonymous

May 14, 2026

Abstract

LLM-based agents frequently exhibit *semantic stagnation*—executing superficially diverse but collectively unproductive action sequences during long-horizon tasks. Detecting such stagnation at the *window* level requires labels, yet standard data-programming aggregation (majority vote) assumes labeling functions (LFs) fire independently and identically across time steps, an assumption violated in sequential agent data. We show that LFs on agent trajectories exhibit systematic temporal roles: `command_repetition` leads stagnation onset by $\sim$1.8 steps while `outcome_backpropagation` lags by $\sim$2.1 steps ($p<0.001$). We propose a *temporal label model* (TLM) that incorporates these dynamics via an HMM, improving label quality by 12.2 pp over majority vote and 5.9 pp over FlyingSquid on a 412-window human-annotated gold set (κ=0.82). A lightweight GRU probe, PROGRESSPROBE, trained on TLM labels achieves 0.86 macro-F1 on the gold set, outperforming the LF oracle by 6.0 pp ("student exceeds teacher") and the best LLM-as-judge configuration (Claude-3.5-Sonnet with chain-of-thought) by 4.5 pp while being 690$\times$ faster. In cross-agent evaluation (SWE-agent$\rightarrow$OpenHands), PROGRESSPROBE retains 0.79 F1 ($-$6.7 pp) whereas the LF oracle degrades to 0.58 ($-$21.6 pp), demonstrating that the probe captures generalizable goal-directed progress signals rather than agent-specific heuristics.

1 Introduction

LLM-based agents Yao et al. [2023], Yang et al. [2024], Wang et al. [2024b] have demonstrated impressive capabilities on software engineering, web navigation, and scientific tasks. However, these agents frequently enter periods of *semantic stagnation*: they continue to generate plausible-looking actions—editing files, running commands, searching documentation—yet make no meaningful progress toward their goal. Unlike simple repetition loops, semantic stagnation involves *diverse* actions that are collectively ineffective, making it invisible to cycle detectors and difficult even for the agent itself to recognize.

Detecting stagnation is crucial for cost control, early termination, and adaptive intervention. Existing approaches have significant gaps: (a) cycle detectors Zhuge et al. [2024], Zhang et al. [2024] only catch structural repetition; (b) process reward models Lightman et al. [2023], Wang et al. [2024a] require expensive step-level outcome annotations; (c) LLM-as-judge approaches Zheng et al. [2024] incur high latency and cost; (d) self-reflection mechanisms Shinn et al. [2023], Yao et al. [2024] rely on the stalled model's own judgment.

We formulate progress detection as a *window-level binary classification* problem over sliding windows of k=5 consecutive agent steps, and attack it with weak supervision. Four labeling functions (LFs) provide noisy labels—`command_repetition`, `edit_churn`, `output_diversity`, and `outcome_backpropagation`—but standard majority-vote aggregation yields only 0.72 macro-F1 on our human gold set because 72% of windows fall in the low-confidence region where at most one LF fires.

Our key insight is that LFs on sequential agent data violate the i.i.d. assumption underlying standard data programming. We empirically demonstrate that LFs exhibit systematic *temporal roles*: `command_repetition` is a leading indicator of stagnation onset (-1.82 steps, $p<0.001$), while `outcome_backpropagation` is a lagging indicator ($+2.13$ steps, $p<0.001$). This motivates our *temporal label model* (TLM), an HMM-based aggregation method that explicitly models state transitions and LF emission roles.

Our contributions are threefold:

1. **Temporal label model for sequential data.** We propose TLM, which improves label quality by 12.2 pp over majority vote and 5.9 pp over FlyingSquid Fu et al. [2020] on a human gold set, establishing that time-aware aggregation significantly outperforms i.i.d. methods on sequential weak supervision tasks.
2. **ProgressProbe.** A lightweight GRU probe trained on TLM labels achieves 0.86 macro-F1, surpassing the LF oracle (0.80, "student exceeds teacher") and the best LLM-as-judge (0.81) at $690\times$ lower latency.
3. **Cross-agent transferability.** PROGRESSPROBE transfers from SWE-agent to OpenHands trajectories with only -6.7 pp degradation (F1=0.79), while the LF oracle drops by -21.6 pp (F1=0.58), demonstrating that the probe captures generalizable progress signals.

2 Related Work

Agent monitoring and failure detection. Cycle detection methods Zhuge et al. [2024], Zhang et al. [2024] identify structural repetition in agent trajectories but miss semantically diverse stagnation. TrajAD Li et al. [2024] applies anomaly detection but targets outlier steps rather than sustained unproductive windows. Self-reflection approaches Shinn et al. [2023], Yao et al. [2024], Hao et al. [2023] ask the agent to evaluate its own progress, inheriting the limitations of the stalled model.

Process reward models. PRMs Lightman et al. [2023], Wang et al. [2024a], Uesato et al. [2022] assign step-level correctness scores, typically requiring human annotations per step. Our approach operates at the window level with automatic weak labels, positioning it as a weak-supervision alternative that avoids per-step annotation costs.

Data programming and weak supervision. Snorkel Ratner et al. [2017] introduced programmatic labeling with majority vote and generative label models. FlyingSquid Fu et al. [2020] models pairwise LF correlations via triplet statistics. These methods assume i.i.d. data points. Our temporal label model extends data programming to sequential settings by modeling LF temporal dynamics—leading, concurrent, and lagging emission roles—via a hidden Markov model.

LLM-as-judge. Using LLMs as evaluators Zheng et al. [2024], DuBois et al. [2024] provides flexible assessment but at high computational cost. We benchmark against Claude-3.5-Sonnet and GPT-4o in zero-shot, few-shot, and chain-of-thought configurations, showing that our probe achieves higher accuracy at three orders of magnitude lower cost.

3 Method

3.1 Problem Formulation

Given an agent trajectory $\tau = (s_1, s_2, \ldots, s_T)$ where each step s_t consists of an action and observation, we extract sliding windows $w_i = (s_i, s_{i+1}, \ldots, s_{i+k-1})$ of size k. The task is binary classification: $y_i \in$

LF	Offset	Std	Role	p-value
cmd_rep	-1.82	0.94	Leading	3×10^{-5}
edit_churn	-0.34	1.21	Concurrent	0.14
output_div	$+0.12$	1.08	Concurrent	0.38
outcome_bp	$+2.13$	1.45	Lagging	1×10^{-5}

Table 1: LF temporal roles relative to stagnation onset, measured over 3,847 onset windows. Offset is in steps (negative = before onset).

$\{\texttt{progress}, \texttt{stagnation}\}$. A window is `stagnation` if the actions within it, while potentially diverse at the surface level, collectively fail to advance the agent toward its goal.

3.2 Labeling Functions

We define four LFs that capture complementary signals of stagnation:

- **`command_repetition`**: Detects repeated or near-identical command patterns within a window using token-level similarity.
- **`edit_churn`**: Identifies repeated modifications to the same file regions—edits that are overwritten or reverted.
- **`output_diversity`**: Measures the semantic diversity of command outputs; low diversity suggests the agent is receiving redundant feedback.
- **`outcome_backpropagation`**: Propagates trajectory-level outcome signals backward to label windows near the trajectory's end.

3.3 LF Temporal Dynamics

Standard data programming treats each LF vote as an i.i.d. draw. We hypothesize that on sequential agent data, LFs fire at systematically different offsets relative to stagnation onset. To test this, we align 3,847 stagnation onset windows and measure each LF's mean firing offset.

Table 1 confirms our hypothesis: `command_repetition` fires 1.82 steps *before* stagnation onset (leading indicator; $p<0.001$), while `outcome_backpropagation` fires 2.13 steps *after* (lagging indicator; $p<0.001$). The two concurrent LFs (`edit_churn`, `output_diversity`) fire near onset with non-significant offsets. Granger causality testing confirms that `command_repetition` predicts stagnation onset ($p=0.002$).

3.4 Temporal Label Model

Motivated by these temporal dynamics, we propose the *temporal label model* (TLM), an HMM with two hidden states $\{\texttt{progress}, \texttt{stagnation}\}$. The model learns:

- **Transition probabilities**: $P(\texttt{stag} \mid \texttt{prog}) = 0.083$; $P(\texttt{prog} \mid \texttt{stag}) = 0.142$, capturing the asymmetry that stagnation tends to persist while recovery is more abrupt.
- **Emission model**: Each LF's vote is conditioned on the hidden state and its learned temporal role. Leading LFs receive upweighted influence near state transitions; lagging LFs are downweighted in those regions.

The model is trained via Expectation-Maximization on the full set of LF votes (54,809 windows). Bayesian Information Criterion confirms the temporal model over a static baseline ($\Delta\text{BIC} = -342.7$).

3.5 ProgressProbe Architecture

Each step s_t in a window is encoded into a d-dimensional embedding using Qwen3-Embedding-0.6B (d=1024). The sequence of k=5 embeddings is processed by a single-layer GRU (hidden size 256) followed by an attention-weighted aggregation over hidden states and a linear classifier. The model has 847K parameters and runs in $\sim$3 ms per window. An optional supervised contrastive (SupCon) auxiliary loss encourages separation in hidden space.

4 Experimental Setup

Data. We collect trajectories from SWE-agent Yang et al. [2024] on SWE-bench tasks. After windowing (k=5), we obtain 54,809 windows total. The high-confidence subset (conf$\geq$2: at least 2 LFs agree) contains 15,401 windows. For cross-agent evaluation, we collect 312 OpenHands Wang et al. [2024b] trajectories (8,934 windows).

Human gold set. Three annotators label 412 windows (248 progress, 164 stagnation) with Fleiss' $\kappa = 0.823$ ("almost perfect agreement"), 91% majority agreement, and 74% unanimous agreement. Disagreements concentrate at transition boundaries ($\pm$1 step from stagnation onset/offset). All reported results are evaluated on this gold set unless otherwise noted.

Baselines. We compare against: (1) **Repeat-3**: flags stagnation if $\geq$ 3 of 5 steps contain repeated commands; (2) **CosineSim**: cosine similarity between adjacent step embeddings; (3) **Mean-pool+LR/MLP**: mean-pooled window embeddings with logistic regression or MLP; (4) **Structural features+GBDT**: 18 hand-crafted numerical features; (5) **LF Oracle**: real-time majority vote of all 4 LFs; (6) **FlyingSquid** Fu et al. [2020]: correlation-aware label model; (7) **LLM-as-Judge**: Claude-3.5-Sonnet and GPT-4o in zero-shot, few-shot, and CoT settings.

5 Results

5.1 Main Results on Human Gold Set

Table 2 presents all methods evaluated on the human gold set. PROGRESSPROBE (trained on TLM labels, conf$\geq$2) achieves the best macro-F1 of 0.858, outperforming the LF oracle by 6.0 pp, the best LLM-as-judge (Claude Sonnet CoT) by 4.5 pp, and majority-vote probe by 2.7 pp.

5.2 Label Model Comparison

Table 3 presents the label model ladder. TLM improves over majority vote by 12.2 pp and over FlyingSquid by 5.9 pp on the gold set. The improvement concentrates in the conf=1 region (72% of windows): TLM achieves 0.79 accuracy vs. MV's 0.64 and FlyingSquid's 0.71. FlyingSquid's correlation-aware modeling improves over MV (+6.2 pp) but lacks temporal structure, leaving gains on the table at stagnation boundaries.

The HMM learns interpretable structure: transition probability $P(\texttt{stag} \mid \texttt{prog}) = 0.083$ reflects that stagnation onset is gradual, while $P(\texttt{prog} \mid \texttt{stag}) = 0.142$ reflects more abrupt recovery. Learned emission roles match the empirical LF dynamics (§3.3): `command_repetition` as leading, `edit_churn` and `output_diversity` as concurrent, and `outcome_backpropagation` as lagging.

Method	Gold F1	Latency
Repeat-3	0.589	<1 ms
CosineSim (opt. thresh.)	0.583	<1 ms
Structural GBDT	0.725	<1 ms
MV (label model)	0.720	—
Mean-pool + LR	0.782	2 ms
Mean-pool + MLP	0.796	2 ms
FlyingSquid (label model)	0.782	—
LF Oracle	0.799	—
GPT-4o CoT	0.794	2,140 ms
Claude Sonnet CoT	0.813	2,140 ms
Probe (MV labels)	0.831	3 ms
TLM (label model)	0.841	—
PROGRESSPROBE (TLM labels)	**0.858**	**3 ms**

Table 2: All methods on the 412-window human gold set (κ=0.82). Label model rows show label accuracy directly; probe/baseline rows show model predictions. Latency is per-window inference time (excluding encoding).

Label Model	Gold F1	Conf=1 Acc	Probe $\downarrow$ F1
Majority Vote	0.720	0.64	0.831
FlyingSquid	0.782	0.71	0.831
TLM (Ours)	**0.841**	**0.79**	**0.858**

Table 3: Label model ladder. Gold F1: label accuracy on human annotations. Conf=1 Acc: accuracy on the 72% of windows where ≤ 1 LF fires. Probe $\downarrow$ F1: downstream probe performance on gold set when trained on each label model's outputs.

5.3 Student Exceeds Teacher

A central finding is that PROGRESSPROBE surpasses the LF oracle that generated its training signal. On the gold set, PROGRESSPROBE achieves 0.858 F1 vs. the LF oracle's 0.799 (+6.0 pp). The gap is most pronounced in the conf=1 region where the LF oracle reduces to a single LF's precision (0.64 accuracy), while the probe leverages learned embedding patterns to maintain robust predictions. This "student exceeds teacher" effect confirms that the GRU extracts progress signals from the embedding space beyond what the heuristic LFs encode.

5.4 Comparison with LLM-as-Judge

Table 4 compares PROGRESSPROBE against LLM-as-judge baselines. The best LLM configuration (Claude-3.5-Sonnet with CoT) achieves 0.813 macro-F1—reasonable but below PROGRESSPROBE's 0.858 (-4.5 pp). LLM errors concentrate on two failure modes: (a) classifying "surface-active but goal-divergent" windows as progress, and (b) classifying "brief retreat before refocus" windows as stagnation.

Critically, PROGRESSPROBE operates on the Pareto frontier of cost vs. accuracy: it achieves strictly higher F1 at three orders of magnitude lower cost and latency.

5.5 Cross-Agent Transfer

To test whether PROGRESSPROBE learns generalizable progress signals rather than SWE-agent-specific heuristics, we evaluate the SWE-agent-trained probe zero-shot on 8,934 windows from 312 OpenHands

Model	Method	F1	Lat.	Cost
GPT-4o	Zero-shot	0.729	2.1 s	$4.82/1k
GPT-4o	Few-shot	0.776	2.1 s	$4.82/1k
GPT-4o	CoT	0.794	2.1 s	$4.82/1k
Sonnet	Zero-shot	0.745	2.1 s	$4.82/1k
Sonnet	Few-shot	0.789	2.1 s	$4.82/1k
Sonnet	CoT	0.813	2.1 s	$4.82/1k
PROGRESSPROBE		**0.858**	**3 ms**	**$0.007/1k**

Table 4: LLM-as-judge vs. PROGRESSPROBE. Latency and cost are per window. PROGRESSPROBE is 690× faster and ∼690× cheaper than LLM baselines.

Method	SWE F1	OH F1	Δ
Repeat-3	0.589	0.510	−7.9
LF Oracle	0.799	0.582	−21.6
PROGRESSPROBE	**0.858**	**0.791**	**−6.7**

Table 5: Cross-agent transfer (SWE-agent → OpenHands). PROGRESSPROBE degrades gracefully (−6.7 pp) while the LF oracle collapses (−21.6 pp).

trajectories (Table 5).

PROGRESSPROBE retains 0.791 F1 (−6.7 pp), well above the 0.75 threshold for practical utility. The LF oracle drops catastrophically from 0.799 to 0.582 (−21.6 pp). Per-LF analysis reveals the cause: `command_repetition` degrades to 0.491 F1 and `edit_churn` to 0.385 F1 because these LFs encode SWE-agent-specific action formats. Only `outcome_backpropagation` partially transfers (0.623 F1) due to its more generic signal. The probe, by contrast, operates in the embedding space where surface-level agent differences are abstracted away, enabling transfer.

5.6 Ablation Studies

Temporal modeling is essential. Mean-pool+MLP (no temporal modeling) achieves 0.857 F1 on conf≥2, trailing the GRU probe's 0.919 by 6.1 pp. The gap widens on noisy full data (conf≥1): 0.778 vs. 0.847 (−6.9 pp), suggesting temporal modeling also provides implicit denoising (Table 6).

Window size. k=5 is optimal (0.919 macro-F1). k=3 under-accumulates signal (−3.7 pp, primarily stagnation-F1 −5.3 pp), while k=7 introduces boundary confusion (−0.5 pp) with diminishing returns and higher latency (4.8 ms vs. 3.1 ms).

Encoder choice. We ablate four encoders (all d=1024 except GTE at 768): Qwen3-0.6B (0.919), BGE-M3 (0.903), E5-large (0.898), GTE-base (0.885). The 3.4 pp range indicates moderate but non-critical encoder sensitivity; even the weakest encoder exceeds all non-probe baselines.

LF hold-one-out. Each LF contributes positively. `command_repetition` is most valuable (−3.9 pp when held out), consistent with its leading indicator role. No single LF's removal drops gold F1 below 0.82, indicating healthy redundancy across the LF set.

Outcome leakage sensitivity. Holding out `outcome_backpropagation` from training drops gold F1 by only 1.2 pp (0.846 vs. 0.858). A last-3-windows-only variant of outcome BP shows negligible

Model	Conf≥2	Conf≥1	Gold
Mean-pool + LR	0.844	0.759	0.782
Mean-pool + MLP	0.857	0.778	0.796
GRU + mean-pool	0.902	—	—
Transformer (4L)	0.908	—	—
GRU + attention	**0.919**	**0.847**	**0.858**

Table 6: Temporal vs. non-temporal architectures. GRU+attention consistently outperforms non-sequential models by 6–7 pp.

further loss (-0.4 pp). The signal is largely redundant with leading/concurrent LFs, confirming no meaningful information leakage.

Multi-seed stability. Across seeds $\{42, 123, 456\}$, conf$\geq$2 macro-F1 is 0.919 ± 0.011. All seeds exceed every baseline by ≥ 5 pp (p=0.008, paired t-test vs. Repeat-3).

Data scaling. At 10% of training data (1,540 windows), PROGRESSPROBE already achieves 0.823 F1. The diminishing-returns knee is at $\sim$40%; 50%$\rightarrow$100% adds only 1.6 pp.

5.7 Non-NLP Structural Baseline

To validate that NL embeddings are necessary, we train classifiers on 18 purely structural features (lines changed, unique command ratio, test execution count, etc.). The best structural model (GBDT) achieves 0.725 macro-F1—19.4 pp below PROGRESSPROBE. Combining structural and NL features yields only +0.2 pp over NL alone (0.920 vs. 0.919), confirming that the embedding subsumes structural signals.

5.8 Error Analysis

On the gold set, PROGRESSPROBE makes 57 errors: 24 false positives and 33 false negatives.

False positives (n=24) are dominated by *exploratory divergence before productive pivots*—short bursts of exploration that the probe marks as stagnation but annotators consider healthy problem-solving. These are inherent to the window-level granularity.

False negatives (n=33) are dominated by *semantic stagnation with diverse tokens*—the agent performs substantively identical operations using different surface forms (varied commands/file paths achieving the same effect). 42% of FN overlap with LF oracle errors, indicating these are intrinsically hard cases. The remaining 58% are unique to the probe, representing its primary limitation.

Error rates are higher on unresolved trajectories (0.19) than resolved ones (0.11), reflecting the greater ambiguity of trajectories where even the overall outcome is failure.

5.9 Interpretability

GRU hidden states projected via UMAP show two distinct clusters (progress/stagnation) with silhouette score 0.63 (0.71 with SupCon). Attention weights concentrate on later steps within each window (last 2 steps receive $\sim$59% of total attention), consistent with the model waiting for sufficient evidence before committing to a classification. At stagnation onset windows specifically, attention peaks at steps 4–5, where repetitive patterns crystallize.

5.10 Application: Early Termination

We simulate an early termination policy on 487 held-out trajectories: terminate when PROGRESSPROBE predicts N consecutive stagnation windows. At the optimal threshold (N=4), this saves 27.4% of tokens while preserving 96.8% of successful resolutions (false termination rate: 3.2%). At the conservative setting (N=5), token savings are 22.8% with 97.9% success preservation.

6 Discussion

Temporal label models beyond agent monitoring. While we demonstrate TLM on agent trajectories, the principle—that LFs on sequential data exhibit temporal roles violating i.i.d. assumptions—applies broadly. Any weak supervision task on time series, event logs, or dialogue where labeling signals have leading/lagging dynamics could benefit from temporal-aware aggregation.

Limitations. (1) Cross-agent transfer is evaluated on two SE agents (SWE-agent, OpenHands); transfer to non-SE domains (web navigation, scientific agents) remains untested. (2) The current LF set is hand-crafted; automatic LF discovery could improve coverage. (3) Window-level classification cannot distinguish productive exploration from aimless wandering within a single window—finer-grained or hierarchical formulations may address this. (4) Our gold set, while large enough for reliable evaluation (κ=0.82, n=412), covers one agent domain.

7 Conclusion

We introduced the temporal label model, a time-aware extension of data programming that models LF temporal dynamics (leading/concurrent/lagging roles) via an HMM. On a human-annotated gold set (κ=0.82), TLM improves label quality by 12.2 pp over majority vote and 5.9 pp over FlyingSquid. PROGRESSPROBE, a lightweight GRU probe trained on TLM labels, achieves 0.86 macro-F1—surpassing the LF oracle by 6.0 pp ("student exceeds teacher") and LLM-as-judge (CoT) by 4.5 pp at 690× lower latency. Cross-agent evaluation demonstrates that the probe captures generalizable progress signals (-6.7 pp transfer degradation vs. -21.6 pp for the LF oracle). A simulated early-termination policy based on probe predictions saves 27% of tokens while preserving 97% of successful outcomes.

References

Ratner, A., Bach, S., Ehrenberg, H., Fries, J., Wu, S., and Ré, C. Snorkel: Rapid training data creation with weak supervision. *Proc. VLDB*, 11(3):269–282, 2017.

Fu, D., Chen, M., Sala, F., Hooper, S., Fatahalian, K., and Ré, C. Fast and three-rious: Speeding up weak supervision with triplet methods. In *Proc. ICML*, 2020.

Lightman, H., Kosaraju, V., Burda, Y., Edwards, H., Baker, B., Lee, T., Leike, J., Schulman, J., Sutskever, I., and Cobbe, K. Let's verify step by step. *arXiv:2305.20050*, 2023.

Wang, P., Li, L., Shao, Z., Xu, R., Dai, D., Li, Y., Chen, D., Wu, Y., and Sui, Z. Math-Shepherd: Verify and reinforce LLMs step-by-step without human annotations. In *Proc. ACL*, 2024.

Yang, J., Jimenez, C., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., and Press, O. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv:2405.15793*, 2024.

Wang, X., Li, B., Song, Y., Xu, F., Tang, X., Zhuge, M., Pan, J., Song, Y., Li, B., et al. OpenDevin: An open platform for AI software developers as generalist agents. *arXiv:2407.16741*, 2024.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. ReAct: Synergizing reasoning and acting in language models. In *Proc. ICLR*, 2023.

Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., and Yao, S. Reflexion: Language agents with verbal reinforcement learning. In *Proc. NeurIPS*, 2023.

Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T., Cao, Y., and Narasimhan, K. Tree of thoughts: Deliberate problem solving with large language models. In *Proc. NeurIPS*, 2024.

Hao, S., Gu, Y., Ma, H., Hong, J., Wang, Z., Wang, D., and Hu, Z. Reasoning with language model is planning with world model. In *Proc. EMNLP*, 2023.

Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Proc. NeurIPS*, 2024.

DuBois, Y., Li, C.X., Taori, R., Zhang, T., Gulrajani, I., Ba, J., Guestrin, C., Liang, P., and Hashimoto, T. AlpacaFarm: A simulation framework for methods that learn from human feedback. In *Proc. NeurIPS*, 2024.

Uesato, J., Kushman, N., Kumar, R., Song, F., Siegel, N., Wang, L., Creswell, A., Irving, G., and Higgins, I. Solving math word problems with process- and outcome-based feedback. *arXiv:2211.14275*, 2022.

Zhuge, M., Zhao, C., Ashley, D., Wang, W., Khizbullin, D., and Schmidhuber, J. Agent workflow memory. *arXiv:2409.07429*, 2024.

Zhang, Y., Lin, H., and Xie, T. SpecRA: Speculation-based runtime analysis for LLM agent loops. *arXiv preprint*, 2024.

Li, Y. et al. TrajAD: Trajectory anomaly detection in LLM agents. *arXiv preprint*, 2024.

A Supplementary Results

A.1 Full Architecture Ablation

Table 7 presents the full architecture comparison.

Architecture	F1	Params	Lat.
MLP on pooled	0.857	—	2 ms
GRU + mean-pool	0.902	—	3 ms
Transformer (4-layer)	0.908	2.13M	4 ms
GRU + attention	**0.919**	**847K**	**3 ms**

Table 7: Architecture ablation on conf$\geq$2 test set.

Encoder	Dim	F1	Enc. Time
GTE-base	768	0.885	28 ms
E5-large	1024	0.898	42 ms
BGE-M3	1024	0.903	38 ms
Qwen3-0.6B	**1024**	**0.919**	**35 ms**

Table 8: Encoder ablation. Macro-F1 on conf$\geq$2 test set.

Held-Out LF	Gold F1	Δ
None (full)	0.858	—
`outcome_bp`	0.846	-1.2
`output_diversity`	0.845	-1.3
`edit_churn`	0.837	-2.2
`command_repetition`	0.820	-3.9
All except `cmd_rep`	0.785	-7.4

Table 9: LF hold-one-out ablation on gold set.

A.2 Encoder Ablation

A.3 LF Hold-One-Out

A.4 CosineSim Threshold Sweep

CosineSim baseline achieves 0.313 F1 at the default threshold of 0.5, and 0.583 at the optimized threshold of 0.72. The score distributions for progress and stagnation windows overlap heavily (overlap coefficient 0.61; KS test p=0.003), confirming that step-pair cosine similarity cannot capture window-level progress signals.

A.5 Full-Data (Conf$\geq$1) Training

Training on the full 54,809 windows (including noisy conf=1 labels, estimated noise rate 22%) yields 0.847 macro-F1, a 5.9 pp drop from the conf$\geq$2 model. Stagnation-F1 is disproportionately affected (-8.2 pp vs. -3.5 pp for progress-F1), reflecting asymmetric label noise where stagnation is more often mislabeled as progress.

A.6 Data Scaling Curve

Train %	Windows	Macro-F1
10%	1,540	0.823
25%	3,850	0.871
50%	7,700	0.902
100%	15,401	0.919

Table 10: Data scaling on conf$\geq$2. Diminishing returns at $\sim$40%.

A.7 SupCon Auxiliary Loss

Adding supervised contrastive loss (weight=0.1, temperature=0.07) improves macro-F1 by 0.6 pp (0.925 vs. 0.919) and substantially improves hidden-state separability (silhouette score 0.71 vs. 0.58).

A.8 Early Termination Details

N	**Tokens Saved**	**Success Pres.**	**False Term.**
3	31.2%	94.7%	5.3%
4	27.4%	96.8%	3.2%
5	22.8%	97.9%	2.1%

Table 11: Early termination simulation on 487 held-out trajectories. N: consecutive stagnation windows required to trigger termination.